

dr. Matej Šprogar

Organizacija podatkov

Preprosto delo s kompleksnimi podatki



Zmožnosti podatkovnih tipov

Do sedaj smo se ubadali z organizacijo kode s pomočjo programskih struktur, podatkovno gledano pa smo omejeni na delo z nekaj tipi števil ali črko preko spremenljivke osnovnega podatkovnega tipa.

Ampak v človeškem svetu niso ključne številke, mi namreč delamo z obilico dokumentov, slik, avtomobilov, ljudi... Potrebujemo nekaj več kot zgolj *integerje*.

Jeziki nam zato ponujajo še *naštevne tipe, polja, razrede (strukture*)*...

Kako shranimo podatek?

Podatek je *katerakoli* vsebina, ki jo želimo shraniti ali obdelati z računalnikom. Ker pomnilnik hrani samo števila, moramo vsak podatek nekako spremeniti v številko in številko tudi pravilno razumeti, ko jo preberemo. Za cela in realna števila ter črke to delo opravijo vgrajeni tipi. Logične vrednosti so prvi primer preslikave podatka v število. Kaj pa kompleksnejše vsebine?

Ura dneva se na primer lahko direktno preslika v število med 0 in 23, čeprav tudi to ni idealno (ali sta recimo -3 in 27 veljavni uri?), kaj pa dan v tednu...

Naštevni tipi

Določeni podatki imajo majhne zaloge vrednosti (dan v tednu, smeri neba, imena barv...). Dodelitev zaporedne številke vsakemu od njih je enostavna rešitev, če bi le lahko nekako **preprečili uporabo napačnih števil** ali računov...

Naštevni tip nam daje možnost, da ustvarimo **nov podatkovni tip** z znano zalogo vrednosti. Spremenljivke tega tipa lahko zavzamejo (v Javi izključno) eno od naštetih vrednosti, lahko jih primerjamo, ne pa tudi seštevamo...

Ročna preslikava podatkov v *int*-e je vedno mogoča, a kar kliče po napaki. Zato namesto

```
const int srce=1, karo=2, pik=3, kriz=4;
int barva = srce;
int nekaj = barva + pik;                // LEGALNO 😬
```

raje uporabimo:

```
enum BarvaKart {srce, karo, pik, kriz};
BarvaKart barva = BarvaKart.srce; // C++ srce
BarvaKart kaj = barva + BarvaKart.pik; // NAPAKA 😬
```

Sedaj prevajalnik skrbi, da s spremenljivko tipa *Barva* ne delamo "neumnosti".

Uporaba naštevni tipov

V Javi in C# definiramo *enum*-e na ravni podprogramov.

Tipične operacije za spremenljivke naštevne tipa so *priveditve* in preverjanje enakosti/različnosti.

C++ in C# ponujata enostavno ročno dodelitev konstant vsaki vrednosti (privzeto od 0 navzgor), v Javi je to bolj zapleteno* (obravnavanje naštevni tipov kot celih števil odpira nove možnosti in tudi nevarnosti).

```
Barva dolociBarvo(BarvaKart b) {  
    if (b == BarvaKart.srce ||  
        b == BarvaKart.karo)  
        return Barva.rdeca;  
    return Barva.crna;  
}
```

C++ in C# obravnavata *enum* kot celo število in dovoljujeta širšo zalogo vrednosti, Java ne:

```
enum Smer {levo=1, desno, gor=4, dol=8};  
  
Smer nekam = Smer.levo | Smer.gor;  
if (0 != (nekam & Smer.levo))  
    System.Console.WriteLine("LEVO!");
```

Večje količine podatkov

Ena spremenljivka nam omogoča delo z *enim* podatkom. Ker pogosto naenkrat rabimo ogromno podatkov, je nemogoče vzdrževati *po eno* spremenljivko za *vsak* podatek. Potrebujemo pristop, ki omogoča enostavno naslavljanje kateregakoli podatka v nekem predelu pomnilnika (podobno, kot poštar uporabi hišno številko za določitev hiše v ulici).

Programerji namesto o ulicah podatkov raje govorimo o *poljih* podatkov, hišne številke pa nadomestimo z *indeksi*...

Polje

Na pomnilnik lahko gledamo kot na velika **polja** (ulice) podatkov, kjer vsak podatek sicer nima svojega lastnega imena, znano pa je *ime* celega polja.

Dostop do polja podatkov izbranega tipa nam omogoča posebna spremenljivka (tim. *referenca*) polja, ki jo najavimo z oglatimi oklepaji:

```
tip[] ime_polja; // Java, C#  
tip *ime_polja;  // C++
```

C++ omogoča tudi spremenljivko polja, ki hrani vse podatke znotraj sebe:

```
tip ime_polja[N]; // C++
```

Delo s polji je **dvofazen** postopek:

1. najprej ustvarimo spremenljivko polja, *preko katere* želimo delati s poljem:
`int[] ulica;`
2. "povežemo" jo s *pravim* poljem izven same spremenljivke.

Spremenljivko/referenco polja potrebujemo za posredni dostop do podatkov v *pravem* polju.

Referenca polja NI polje, ampak spremenljivka, ki hrani informacijo (naslov) za dostop do polja.

Kreiranje polja v Javi in C#

Novo spremenljivko polja lahko s preprosto prireditvijo "povežemo" z:

ali (1) novim poljem, ki ga ustvarimo z operatorjem `new[]`. Novo polje je dostopno izključno preko te spremenljivke:

```
new tip[velikost_polja];
```

ali (2) obstoječim poljem. V tem primeru lahko do istih podatkov dostopamo preko vsaj dveh spremenljivk (stare in nove).

```
int[] ulica; // spremenljivka polja
```

scenarij (1) - Polje **ustvarimo** s klicem systemskega operatorja **`new[]`**, ki rezervira in inicializira pomnilnik:

```
ulica = new int[3];
```

scenarij (2) - Spremenljivko polja lahko inicializiramo s pomočjo že *obstoječega* polja:

```
int[] ista_ulica = ulica;
```

Če spremenljivke polja ne inicializiramo, Java in C# javita napako ob morebitni uporabi.

Dostop do podatkov v polju

Polje je zaporedje (ulica) več podatkov istega tipa, prvi je na (hišni) številki 0, naslednji 1 itd. Tej številki pravimo tudi **indeks**.

Če želimo v programu dostop do konkretnega podatka v polju, moramo *poznati* njegov indeks:

```
ime_polja[indeks];
```

Na ta način *programsko* sestavimo ime podatka v polju.

Šele po povezavi spremenljivke polja z dejanskim poljem lahko dostopamo do podatkov v polju, ki so ***vsi istega*** tipa:

```
int[] ulica = new int[3];  
ulica[0] = 42;  
ulica[1] = 21;  
ulica[2] = 2;
```

Ker je to pogosta kombinacija ukazov, obstajata seveda tudi krajša, a enakovredna ukaza:

```
int[] ulica = new int[]{42, 21, 2};  
int[] ulica = {42, 21, 2};
```

Indeksi

Podatki v polju so indeksirani od 0 naprej. Ker je indeks številka, lahko namesto (literalne) konstante vedno uporabimo tudi spremenljivko ali funkcijo ali izraz, ki vrne celo število.

Če želimo specifičen podatek, moramo **poznati** tudi njegov indeks. Če želimo obdelati vse podatke, moramo poznati **velikost** polja. V C++ imamo `size()`, v Javi `.length` in v C# `.Length`.

Uporaba napačnih indeksov **sesuje*** program.

```
int[] ulica = {42, 21, 2};  
int kje42 = 0;  
int vrne0() { return 0; }
```

sedaj je enakovredno, če uporabimo `ulica[0]`
ali `ulica[3-2-1]` ali `ulica[kje42]` ali
`ulica[vrne0()]`.

Če želimo "pregledati" vse podatke v polju moramo vedeti, koliko jih sploh je... Podatek o tem je shranjen v okviru polja:

```
for (int i=0; i<ulica.length; ++i)  
    if (ulica[i] == nekaj) ...
```

Naslovi pomnilnika

Pomnilnik je shramba *bytov* podatkov, ki so lepo postavljeni v vrsto, zato ima vsak svojo unikatno zaporedno številko - **naslov** (lokacijo).

Vsak podatek potrebuje ustrezno velik košček pomnilnika. Če imamo spremenljivko za ta podatek, imamo **neposreden** *read/write* dostop. Sicer moramo do njega dostopati **posredno** preko *reference*.

Spremenljivka **v sebi** hrani nek podatek. Ker uporabljamo njeno ime, ne rabimo njene točne lokacije. Za vse poskrbi prevajalnik.

```
int odgovor = 42;
```

Ta ukaz nekje v pomnilniku za programski sklad rezervira 4 byte, ki jih potem v programu naslavljamo kot `odgovor`; točnega naslova podatka niti **ne rabimo**.



Referenca (Java, C#)

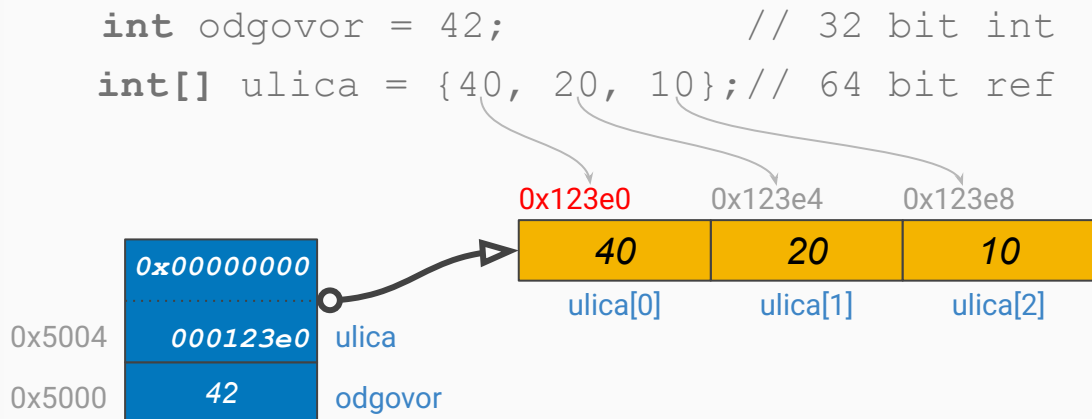
Referenca je* dejanski **naslov** podatka v pomnilniku; spremenljivka, ki hrani referenco, hrani pomnilniški naslov.

Ali v Javi in C# spremenljivka hrani podatek *ali* njegov naslov je določeno s tipom podatka, ki je lahko ali **vrednostne** ali **referenčne** narave.

Vrednostni tipi so vsi osnovni tipi (*int...*), naštevni tipi (*enum*) in *strukture*^(C#). Referenčni tipi so polja *[]* in *razredi*^{*}.

Java in C# zaradi *varnosti* prikrivata uporabo referenc (C#: `func(ref int)`)

Spremenljivka vrednostnega tipa hrani podatek (njegovo vrednost) znotraj *svojega* prostora, medtem ko spremenljivka referenčnega tipa v sebi hrani naslov poljubne *zunanje* lokacije z vrednostjo podatka. Za delo z *enim* podatkom rabimo DVA kosa pomnilnika → dvodomnost!



Sklad in *dinamični pomnilnik (free store)*

Java in C# uporabljata pri poljih drugačen način dela s pomnilnikom kot pri navadnih spremenljivkah. To je povezano z zasnovo jezika, ki temelji prav na delu s pomnilnikom.

Pomnilnik, ki pripada programu, je razdeljen na dva dela: **sklad** (stack) in **dinamični pomnilnik** (free store/heap).

Tim. *referenco* polja v **prvi fazi** ustvari prevajalnik na *skladu*, "pravo" polje pa operator **new** v **drugi fazi** v *dinamičnem pomnilniku*.

Prevajalnik uporablja kratkoživi sklad - prostor za naše spremenljivke se izredno hitro **dodeli** in **sprosti** glede na izvajalni *kontekst*.

Če pa želimo dolgoživ in velik kos spomina, ki se ohrani tudi po koncu bloka, uporabimo **new** za počasno dodelitev *dinamičnega pomnilnika*:

```
int[] ulica = new int[3];
```



C++ referenca ni C#/Java referenca!

C++ *lahko* deluje ali enodomno s poljem na skladu ali dvodomno s kazalci. V slednjem primeru Javanski / C# **referenci** ustreza C++ **kazalec**:

Java in C# referenca $\equiv$ C++ kazalec

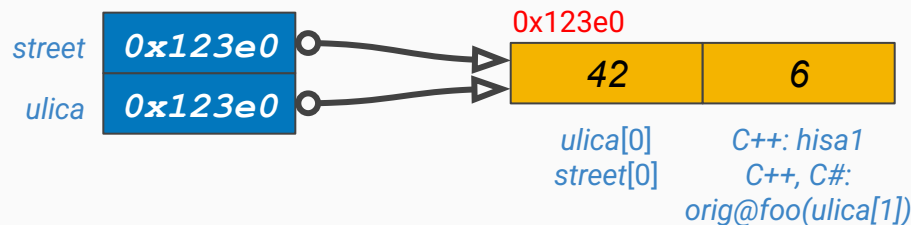
`int[] polje  $\equiv$  int *polje`

C++ referenca (&) je novo ime (*alias*) obstoječe spremenljivke, javanska in C# referenca pa je *nova* spremenljivka.

Java nima ekvivalenta C++ referenci, medtem ko ima C# *alias* le v primeru *ref* funkcijskih argumentov...

```
// C++
int *ulica = new int[]{42,6}; // enodomno: int ulica[]={42,6};
int *street = ulica;          // street == ulica, street!≡ulica
int &hisa1 = ulica[1];        // hisa1 ≡ ulica[1]
delete[] ulica;               // OBVEZNO če new[]!
```

```
// JAVA, C#
int[] ulica = {42, 6};
int[] street = ulica;        // street==ulica, street!≡ulica
```



// klic funkcije, ki lahko spremeni ulica[1]:		
// C++	C#	Java
<code>foo(ulica[1]);</code>	<code>foo(ref ulica[1]);</code>	<code>foo(ulica, 1);</code>
<code>void foo(int& orig)</code>	<code>void foo(ref int orig)</code>	<code>void foo(int[], int)</code>
<code>{}</code>	<code>{}</code>	<code>{}</code>

Kopija in enakost

Za kopiranje in primerjavo vsebine polj v Javi in C# žal ne smemo uporabiti namenskih operatorjev za pisanje (=) in primerjavo (==), ampak posebne funkcije, ki znajo delati z referencami in polji!

Zaradi tega in ker poljem ne moremo spremeniti velikosti, je sicer pametneje uporabljati specializirane razrede* *ArrayList<>* (Java) in *List<>* (C#), ki opravijo isto nalogo bolje. Več o vsem tem v okviru *podatkovnih struktur*.

C	1972	<pre>int ulica[] = {42, 21, 2}; int cesta[sizeof(ulica)/sizeof(int)]; memcpy(cesta, ulica, sizeof(ulica)); preveri(0 == memcmp(ulica, cesta, sizeof(ulica)));</pre>
Python	1991	<pre>ulica = [42, 21, 2] cesta = ulica.copy() preveri(ulica == cesta)</pre>
C++	1994	<pre>vector<int> ulica = {42, 21, 2}; vector<int> cesta = ulica; preveri(ulica == cesta);</pre>
Java	1995	<pre>int[] ulica = {42, 21, 2}; int[] cesta = ulica.clone(); preveri(Arrays.equals(ulica, cesta));</pre>
C#	2000	<pre>int[] ulica = {42, 21, 2}; int[] cesta = (int[])ulica.Clone(); preveri(ulica.SequenceEqual(cesta));</pre>
Java	2004	<pre>ArrayList<Integer> ulica = new ArrayList<>(Arrays.asList(42, 21, 2)); ArrayList<Integer> cesta = new ArrayList<>(ulica); preveri(ulica.equals(cesta));</pre>
C#	2005	<pre>List<int> ulica = new List<int>{42, 21, 2}; List<int> cesta = new List<int>(ulica); preveri(ulica.SequenceEqual(cesta));</pre>

Kopiranje in primerjava polj?

Razlika med vrednostno in referenčno spremenljivko določa kopiranje in primerjavo polj. Referenca je naslov, torej številka, zato pri kopiranju in primerjavi referenc **ne kopiramo** in **ne primerjamo** referenciranih vsebin, ampak zgolj reference!

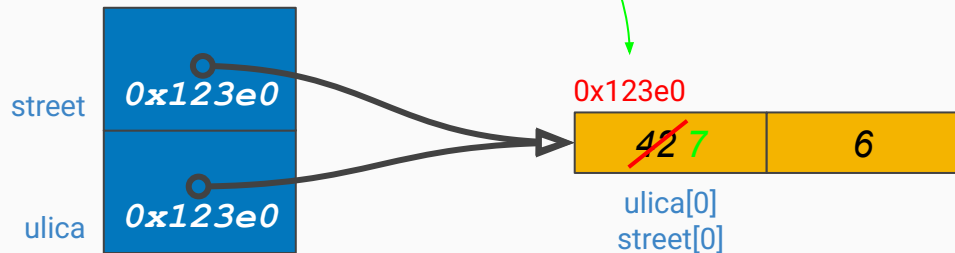
Ker programiramo s spremenljivkami, dejansko delamo z referenco podatka, ne pa z njegovo vsebino!

```
int[] ulica = {42, 6};  
int[] street = ulica;           // C#, Java
```

```
preveri(street == ulica);      // 0x123e0 == 0x123e0  
preveri(street[0] == ulica[0]); // 42 == 42
```

```
street[0] = 7;                 // spremeni ulico!
```

```
preveri(street == ulica);      // 0x123e0 == 0x123e0  
preveri(street[0] == ulica[0]); // 7 == 7
```



1. Spremenljivka polja je v Javi in C# **vedno** referenca, ne glede na vsebino polja.
2. Kopiranje in primerjava Java in C# reference **ne** pomeni kopiranja in primerjave **vsebine** polja.

Lastnosti polj

- + enostavna za uporabo
- + hitra
- nespremenljive velikosti
- vsi podatki istega tipa

Problem nespremenljive velikosti rešijo *List<>* (#), *ArrayList<>* (Java) in *vector<>* (C++).

Tipiziranost podatkov ni težava, ampak prednost.

```
int[] p = new int[50]; // 50 elementov fiksno!
```

```
p[3] = 12; // enostavno, hitro!
```

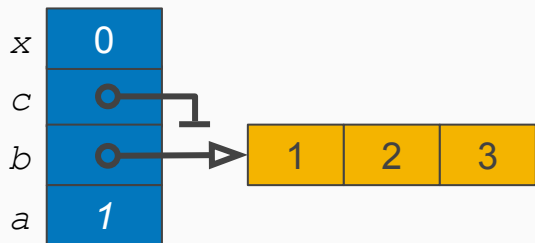
```
p[50] vrže izjemo in ubije program  
p.povečaj_kapaciteto(51) žal ne obstaja
```

```
ArrayList<Integer> arr = new ArrayList<Integer>();  
arr.add(77);  
arr.add(42);
```

```
p[0] = "abc" je (hvala bogu!) sintaktična napaka,  
čeprav lahko to ukažemo recimo v Pythonu ali  
Javascriptu...
```

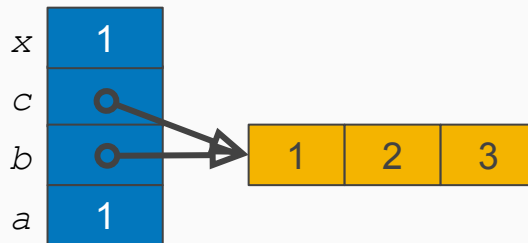
Razlika med referenčnimi in vrednostnimi spremenljivkami

```
int a = 1;  
int[] b = {1,2,3};  
int[] c /* = null */;  
int x;
```



```
int a = 1;  
int[] b = {1,2,3};  
int[] c;  
int x;
```

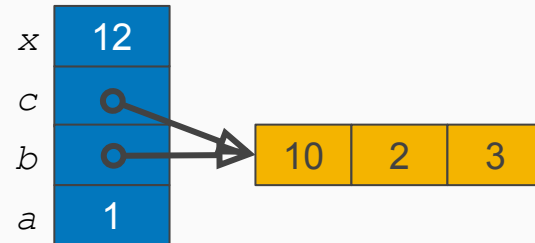
```
c = b;  
x = a;
```



```
int a = 1;  
int[] b = {1,2,3};  
int[] c = b;  
int x = a;
```

```
c[0] = 10;  
preveri(b[0] == 10);
```

```
x = 12;  
preveri(a == 1);
```



Večdimenzionalna polja

Za dostop do podatka v polju potrebujemo (1) ime polja in (2) pozicijo podatka. Kot indeks lahko *uporabimo* tudi spremenljivko, samo ime polja pa je fiksno 😞 del programske kode, ki ne gre v spremenljivko. To je neugodno, če moramo recimo vzdrževati in delati z *dosti* polji.

Ena rešitev je **polje polj** (`[ ][ ]`), kjer so elementi spet(!) polja in jih ne rabimo poimenovati, saj so oštevilčena. Prvi (levi) indeks predstavlja oznako podpolja, zadnji (desni) pa oznako elementa v podpolju.

Polje polj ...

Dvo/več-dimenzionalno polje hrani reference na (pod)polja. Slednja ne potrebujejo imen, ampak so programsko dostopna preko indeksov.

Dostop do podatka je mogoč ali (1) neposredno z več indeksi:

```
mesto[id_ulice][id_hiše]
```

ali (2) posredno preko nove reference na podpolje:

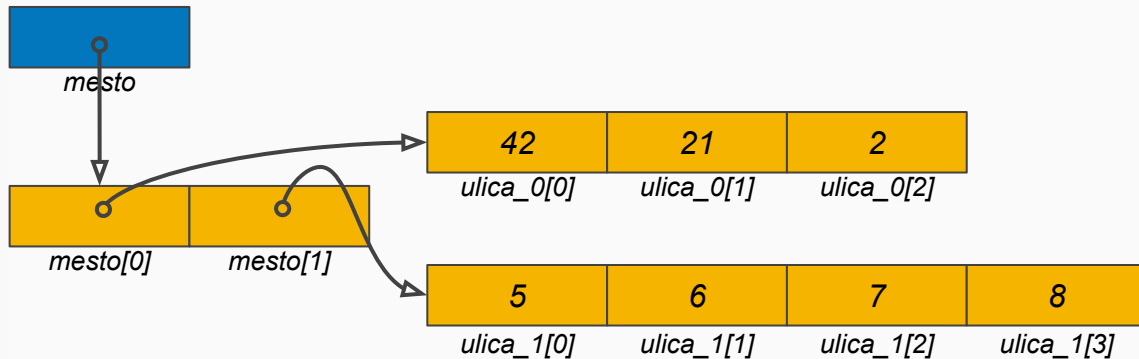
```
int[] ulica = mesto[id_ulice];  
ulica[id_hiše]
```

Ideja je razširljiva na še več dimenzij.

```
int[][] mesto = new int[2][];  
mesto[0] = new int[]{42, 21, 2};  
mesto[1] = new int[]{ 5, 6, 7, 8};
```

oziroma v krajši, a enakovredni obliki:

```
int[][] mesto = {  
    new int[]{42, 21, 2},  
    new int[]{ 5, 6, 7, 8}  
};
```



C#

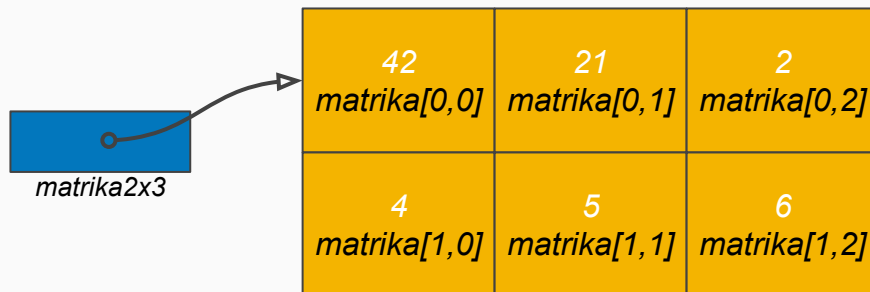
Pravokotna polja

Pravokotna polja so poseben primer večdimenzionalnih polj, kjer so vsa podpolja **enake velikosti**. C# za njih ponuja posebno notacijo z vejico namesto z večkratnimi oklepaji.

Tudi pravokotna polja lahko razširimo čez več kot dve dimenziji; najlažje si jih predstavljamo kot matriko (2D) oziroma (hiper)prostor (3D+).

```
int[,] matrika = new int[,] {  
    {42, 21, 2},      // prva vrstica  
    {4, 5, 6}         // druga vrstica  
};  
  
int odgovor = matrika[0,0];  
int[] vrstica = matrika[0];      // NAPAKA
```

Java in C++ nimata te notacije, dosežeta pa enak učinek z večdimenzionalnim poljem:
(C# [a,b] == JavaCpp[a][b]).



String

Polje je *osnova* za eno pomembnejših podatkovnih struktur - **niz znakov**. C++ in C# ga imenujeta *string*, Java *String*. Delo s stringi (lepljenje, primerjave*, izrezovanje, iskanje podnizov...) je mnogo lažje od dela s polji znakov.

Za zapis string konstante uporabimo par dvojnih narekovajev:

```
"string", "dve besedi".
```

String "A" NI enako kot črka 'A'.

Prazen string "" NI `null` referenca.

Osnovna operacija pri delu z nizi znakov je primerjava, ki seveda poteka preko referenc:

```
String ena = "abc";           // ≈ {'a','b','c'}  
String dva = "abc";           // dva = ena;  
preveri(ena == dva);          // Java referenca
```

```
String tri = new String("abc"); // Java, C#  
preveri_Java(ena != tri);        // 😱  
preveri_Java(ena.equals(tri));   // 😐  
preveri_Cpp_Cis(ena == tri);     // 😎
```

C++ in C# operator `==` služita tudi za primerjavo stringov po vsebini, kar omogoča bolj "domačo" uporabo stringov v kodi.

Operacije s stringi

String hrani poljubno dolge nize znakov, vključno s posebnimi znaki. Sestavimo ga najlažje z lepljenjem več stringov s pomočjo `+` in `+=`.

Primerjave so mogoče preko operatorja `==` oz. Java `.equals`. Vrstni red lahko določimo s pomočjo metode `.compare` (C++), `.compareTo` (Java) in `.CompareTo` (C#).

Uporaba je intuitivna, na voljo so metode za pretvorbo med velikimi in malimi črkami, v polje znakov...

Lepljenje stringov je enostavno tudi v Javi:

```
String pozdrav = "Halo, ";  
String oseba = "svet";  
  
String stavek = pozdrav + oseba;  
stavek += "!\n";  
preveri(stavek.equals("Halo, svet!\n"));  
  
preveri("abc".equals("ABC".ToLower()));  
preveri("abc".CompareTo("ABC") < 0);
```

Java sicer ne omogoča indeksnega naslavljanja:

```
preveri_Cpp_Cis("abc"[0] == 'a');  
preveri_Java("abc".charAt(0) == 'a');
```

Zakulisje...

String je izredno pomemben tip, ki *mora* omogočati hitro in varno kodo. Java in C# ga hranita kot polje znakov s pomočjo reference, kar avtomatsko pomeni nevarnost, saj **pošiljanje reference v tujo funkcijo vedno omogoča tej funkciji, da spremeni "naše" podatke.**

Ko pošlje referenco lahko klicajoča koda samo "upa", da klicana funkcija ne bo spremenila referencirane vsebine - razen če bi bili vsi poskusi spremembe onemogočeni...

Java in C# zato uporabljata *nespremenljive* (*immutable*) stringe - vsaka operacija, ki bi spremenila string, rezultira v **novem** stringu, original ostaja nedotaknjen:

<u>koda:</u>	→	<u>zakulisna koda:</u>
<code>string s = "abc";</code>		<code>string s = "abc";</code>
<code>s += "d";</code>		<code>string s1 = s + "d";</code>
<code>s += "e";</code>		<code>string s2 = s1 + "e";</code>

Stringe lahko sedaj pošiljamo v druge funkcije brez skrbi, da bodo spremenile original - dve različni funkciji lahko varno *hkrati* uporabita (bereta!) isti string, kar je nepogrešljivo pri paralelnem izvajanju kode. Ta počasna rešitev sicer poveča varnost uporabe *stringov*, ne velja pa za druge (naše) referenčne tipe...

Razred

Sedaj znamo delati z veliko količino *osnovnih* podatkov. Problem ostajajo kompleksni podatki, ki zahtevajo (pre)več komplikacij v programski kodi.

Kompleksnejši podatek recimo obsega ne zgolj eno, ampak vsaj dve (ali več) števili. Recimo 2D+ koordinata, kompleksno število, datum... "Najtežavnejši" pa so podatki, ki kombinirajo različne tipe osnovnih podatkov. Recimo račun, stranka, hladilnik, avto, naročilo, mesto, ...

Rešitev zahteva posebno poglavje o razredih.

Naloge

Funkcija, ki ubesedi številko, ver 2.

Preveri imutabilnost stringov!

Funkcija, ki izračuna povprečno vrednost števil v polju? Kaj, če *morajo* števila ustrezati nekemu kriteriju?

Preštej vse črke v stringu!

```
double povprecje(int[] zaloga)
{
    // predpogoj: arhitekturna predpostavka
    assert zaloga != null && zaloga.length > 0;

    // zahtevana vsebinska varovalka
    if (!kriterij(zaloga))
        ustavi_program();

    long vsota = 0;
    for (int i = 0; i < zaloga.length; ++i)
        vsota += zaloga[i];

    return (double)vsota / zaloga.length;
}
```